

Propuesta de tema Laboratorio y Programación 6to año

Proyecto Final de Ingeniería - Propuesta de Tema



Videojuego de carreras 2D multijugador en red local

Fecha: 28/09/2026

<i>Nombre y apellido:</i>
Joaquin Barreira
Madai Andacaba
Antonella Vivacqua
Joaquin Rodrigues
Mateo Guinsburg

Tabla de Contenidos

1 INTRODUCCIÓN 3

 1.1 Objetivo..... 3

 1.2 Alcance 3

2 DESCRIPCIÓN DE LA PROPUESTA..... 4

 2.1 Tácticas de trabajo 5

 2.2 Gráficos o bocetos ilustrativos 5

 2.3 Otros aspectos a tener en cuenta 6

3 APORTES 7

4 APROBACIÓN 7

1 INTRODUCCIÓN

1.1 Objetivo

El objetivo del presente trabajo final es diseñar, desarrollar e implementar un videojuego de carreras en dos dimensiones (2D), de estilo Fórmula 1 arcade, en el que entre dos y cinco jugadores compiten simultáneamente, cada uno desde su propia computadora, a través de una red local (LAN). El juego adopta una estética visual de tipo arcade retro / pixel art, con vista cenital (top-down).

El programa consiste en una aplicación cliente-servidor. El módulo servidor coordina la partida, simula la carrera y sincroniza el estado del juego entre los participantes; el módulo cliente, ejecutado por cada jugador, renderiza la pista, los vehículos y la interfaz, captura los controles del usuario y los envía al servidor. Uno de los jugadores crea la partida y actúa como anfitrión (host): en su computadora se ejecutan el servidor y su propio cliente. Los demás jugadores se unen indicando la dirección IP del anfitrión.

En cada partida los jugadores eligen su piloto y el anfitrión elige el circuito. Gana quien complete primero la cantidad de vueltas establecida.

Lenguaje de programación: Java 17 (LTS). Framework: libGDX, con el proyecto generado mediante la herramienta gdx-liftoff y desarrollado en el IDE Eclipse. Comunicación en red: sockets TCP y UDP de la API estándar java.net, sin bibliotecas de red de terceros ni bases de datos, según se detalla en la sección 2. Control de versiones y documentación: Git y GitHub (repositorio, wiki y registro de cambios).

1.2 Alcance

- Menú principal con las opciones Crear partida, Unirse a una partida (ingresando la dirección IP del anfitrión), Prueba local y Salir.
- Sala de espera (lobby) para entre dos y cinco jugadores, donde cada uno ve a los demás conectados, elige su piloto e indica que está listo. El anfitrión elige el circuito e inicia la carrera.
- Cinco pilotos seleccionables, cada uno con nombre, auto y color propios, sin repetirse dentro de una misma partida. Las diferencias entre pilotos son solo estéticas, para que ninguno tenga ventaja.
- Tres circuitos seleccionables con estética de pista arcade: línea de largada y meta, banquetas, zonas de pasto que reducen la velocidad y público en las tribunas como fondo decorativo.
- Física de movimiento arcade (aceleración, frenado, giro y derrape leve) y colisiones de los vehículos contra los límites de la pista y entre sí.
- Sincronización en red, en tiempo real, de la posición, la orientación y el estado de todos los autos.
- Carrera con cuenta regresiva de largada y sistema de vueltas validado por puntos de control (checkpoints), que impide sumar vueltas tomando atajos. La carrera finaliza al completarse la cantidad de vueltas establecida (tres por defecto).
- Interfaz en pantalla (HUD) con la vuelta actual, la posición en carrera y los tiempos.
- Pantalla de resultados con la posición, el tiempo total y la mejor vuelta de cada jugador, con opción de revancha.
- Manejo de desconexiones: si un jugador o el anfitrión abandonan la partida, el resto recibe un aviso y el juego no se interrumpe de forma inesperada.

- Modo de prueba local para dos jugadores en la misma computadora, utilizado para validar la física y las reglas de la carrera sin depender de la red.

Como ampliaciones opcionales, sujetas al tiempo disponible, se consideran: efectos de sonido y música, descubrimiento automático de partidas en la red local (sin ingresar la dirección IP), minimapa y efectos visuales adicionales.

No forman parte del alcance: más de cinco jugadores por partida, rivales controlados por inteligencia artificial, juego a través de Internet o mediante un servidor central, sistemas de progresión o niveles, almacenamiento en bases de datos ni versiones para dispositivos móviles. El proyecto no se organiza en niveles como un juego de plataformas: cada partida es una carrera completa.

2 DESCRIPCIÓN DE LA PROPUESTA

El desarrollo se aborda en dos grandes ejes de trabajo: (a) el motor de juego, responsable del renderizado 2D, la física de movimiento de los vehículos, las colisiones y las reglas de la carrera; y (b) el módulo de red, responsable de conectar a los jugadores y de mantener sincronizado el estado de la partida entre todas las computadoras.

Motor de juego. Se utiliza libGDX como base para el manejo de la ventana, los gráficos, la entrada y los recursos, de modo de no reinventar un motor gráfico completo. Los circuitos se diseñan con el editor de mapas Tiled y se cargan desde archivos .tmx, que además definen las superficies (pista, pasto y muro), los puntos de control y las posiciones de largada. La física de los autos es propia y simple, sin motores de física externos. La lógica de la carrera se mantiene separada del dibujo en pantalla, de modo que el servidor pueda simularla sin depender de la parte gráfica.

Mecánicas y controles. Cada jugador controla su auto con las flechas o con las teclas WASD: acelerar, frenar o retroceder y girar. La carrera comienza tras una cuenta regresiva. Los autos que salen de la pista pierden velocidad en el pasto y rebotan contra los muros. Una vuelta se cuenta solo si el auto atraviesa todos los puntos de control en orden y cruza la línea de meta. La posición de cada jugador se calcula por vueltas completadas, puntos de control alcanzados y distancia al siguiente punto de control.

Arquitectura de red. Se adopta una arquitectura cliente-servidor con servidor autoritativo: el servidor, que se ejecuta en la computadora del anfitrión, es el único que simula la carrera y decide su resultado, lo que evita conflictos de estado entre los clientes. El anfitrión también se conecta a ese servidor como un cliente más, a través de la dirección local 127.0.0.1. La comunicación combina los dos protocolos de transporte según el tipo de información:

- **TCP:** conexión de los jugadores, sala de espera, elección de piloto y circuito, inicio y cuenta regresiva, fin de la carrera, resultados y desconexiones. Son mensajes poco frecuentes que no pueden perderse ni llegar desordenados, por lo que se aprovecha la entrega confiable y ordenada de TCP.
- **UDP:** controles de cada jugador (cliente → servidor) y estado de todos los autos (servidor → clientes), enviados decenas de veces por segundo. Cada paquete reemplaza al anterior, de modo que un paquete perdido queda corregido por el siguiente. UDP evita las demoras por retransmisión de TCP, que en un juego en tiempo real se notan más que una pérdida ocasional.

Cada paquete lleva un número de secuencia para descartar los que llegan atrasados o duplicados.

Los clientes dibujan el estado recibido interpolando entre actualizaciones sucesivas, para lograr un movimiento fluido. La recepción de datos de red se realiza en hilos separados del hilo gráfico, comunicados mediante estructuras de datos seguras para la concurrencia. El protocolo (mensajes, formato de los paquetes y frecuencias de envío) se documentará en detalle en el repositorio del proyecto.

2.1 Tácticas de trabajo

El desarrollo se organiza de forma incremental, validando cada parte antes de sumar la siguiente:

1. Configuración del proyecto con gdx-liftoff, repositorio en GitHub, wiki y registro de cambios.
2. Estructura de pantallas y navegación (menú, sala de espera, carrera y resultados).
3. Carrera local de un jugador: física del auto, primer circuito y colisiones.
4. Reglas de la carrera con dos autos controlados localmente en la misma computadora, para probar las colisiones, las vueltas y las posiciones.
5. Diseño y documentación del protocolo de red.
6. Conexión y sala de espera por TCP.
7. Carrera sincronizada por UDP.
8. Manejo de desconexiones y errores de red, y pruebas en varias computadoras de la red local.
9. Pilotos, circuitos adicionales y ajustes finales; ampliaciones opcionales si el tiempo lo permite.

El grupo, de cinco integrantes, se divide en tres frentes que trabajan en paralelo: motor de juego (dos integrantes), red (dos integrantes) y arte y documentación (un integrante). Se utiliza Git con una rama principal estable (main), una rama de integración (develop) y ramas por funcionalidad (feature/...). Cada integrante registra sus aportes con commits propios y mensajes descriptivos, y los cambios significativos se documentan en el archivo CHANGELOG.md y en la wiki del repositorio.

2.2 Gráficos o bocetos ilustrativos

Las siguientes figuras muestran el flujo de pantallas del juego, un boceto de la pantalla de carrera y la arquitectura de red. Los recursos gráficos definitivos (sprites de autos y pilotos, tribunas y pista) se diseñarán o adaptarán en estilo pixel art con vista cenital; como material provisorio podrán utilizarse recursos de licencia libre (CC0).

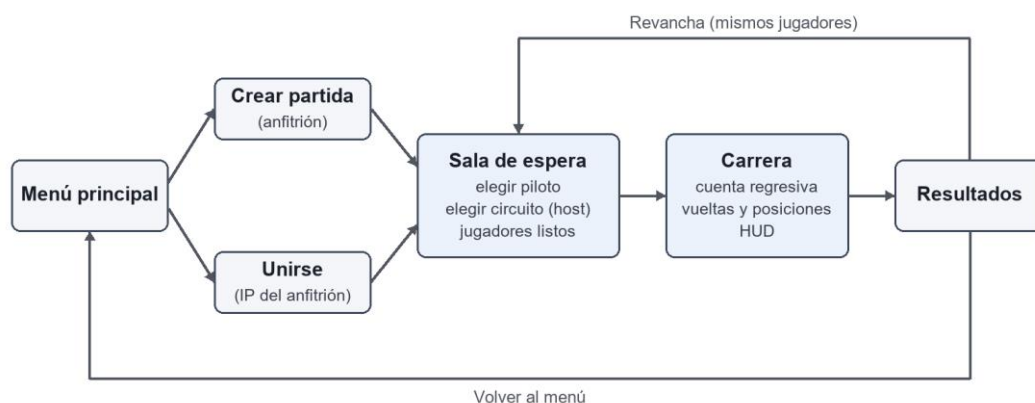


Figura 1. Flujo de pantallas del juego.



Figura 2. Boceto de la pantalla de carrera: vista cenital del circuito y HUD con vuelta, posición y tiempos.

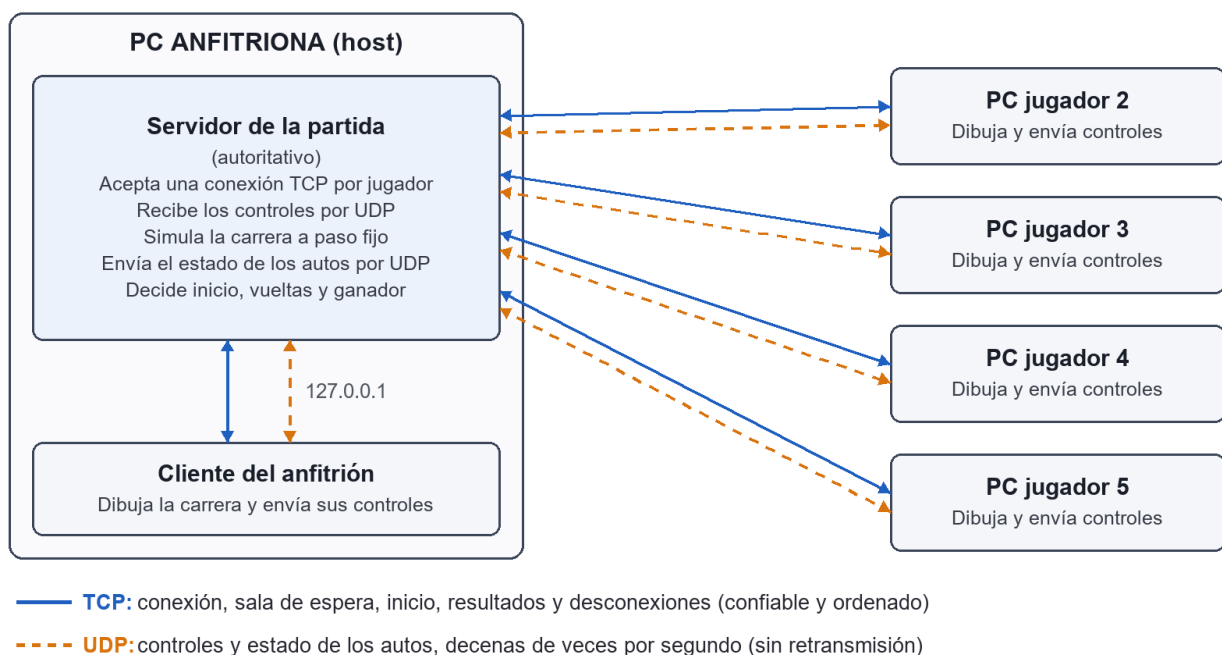


Figura 3. Arquitectura de red: servidor autoritativo en la computadora anfitriona y uso de TCP y UDP según el tipo de información.

2.3 Otros aspectos a tener en cuenta

Requisitos de ejecución. Computadoras con Java 17 o superior conectadas a la misma red local. El juego se distribuye como un archivo .jar ejecutable. Utiliza un puerto TCP y uno UDP configurables (por defecto, 7777 y 7778), que deben estar habilitados en el firewall de la computadora anfitriona.

Riesgos identificados y cómo se mitigan:

- **Restricciones de la red del colegio** (firewall o aislamiento entre equipos): la conexión se probará en la red real desde el comienzo del desarrollo de red. Como alternativa, se utilizará una red propia (router o punto de acceso).
- **Desincronización entre computadoras:** el servidor es la única fuente de verdad del estado de la carrera; los clientes solo envían sus controles y muestran lo que el servidor informa.
- **Pérdida de paquetes UDP:** el diseño tolera pérdidas, y se incluirá una opción de prueba que descarta paquetes a propósito para verificarlo.
- **Plazos:** se prioriza el funcionamiento en red por sobre los agregados. Las ampliaciones opcionales solo se abordan una vez que la carrera en red funcione en varias computadoras.

Los recursos gráficos y sonoros de terceros que se utilicen serán de licencia libre y se acreditarán en el repositorio.

3 APORTES

El principal aporte del trabajo es la integración, en un proyecto acotado y didáctico, de tres componentes que habitualmente se abordan por separado en la formación del alumno: el desarrollo de un juego 2D sobre un framework, la implementación de física y colisiones básicas para un juego de carreras, y la programación de comunicación en red en tiempo real entre varios procesos independientes.

Como mejora respecto de proyectos similares desarrollados en modo local, se propone resolver la sincronización del estado entre hasta cinco jugadores utilizando únicamente la API estándar de Java (java.net), sin motores de red de terceros. Esto constituye un ejercicio concreto de aplicación de los conceptos de programación concurrente y de comunicación por sockets, que incluye la elección fundamentada entre TCP y UDP según el tipo de información transmitida.

Asimismo, se aporta una base de código extensible y documentada (repositorio, wiki y documentación del protocolo), que permite incorporar nuevos circuitos y pilotos con cambios mínimos en el código y que puede servir como punto de partida para futuros trabajos sobre videojuegos en red.

4 APROBACIÓN

Esta deja constancia de la aceptación del tema propuesto.

Firma:

Aclaración:

Fecha: